

Hamming Distance Project report

- a. To come up with this code, I knew I had to create a loop that XOR'ed each bit, and if the output is 1, to increment the counter until the loop was finished. Then I would output the counter as the hamming distance. I also knew I had to write "functions" to handle the conversion from string to ASCII values before I performed any operations on them.

b. Code:

```
section .data
```

```
prompt1 db "Enter the first string: ", 0
```

```
prompt1_len equ $ - prompt1
```

```
prompt2 db "Enter the second string: ", 0
```

```
prompt2_len equ $ - prompt2
```

```
newline db 10
```

```
section .bss
```

```
str1 resb 4 ; For first string
```

```
str2 resb 4 ; For the second string
```

```
output resb 2 ; For ASCII output
```

```
section .text
```

```
global _start
```

```
_start:
```

```
mov eax, 4
```

```
mov ebx, 1 ; stdout
```

```
mov ecx, prompt1 ; print
```

```
mov edx, prompt1_len
```

```
int 0x80
```

; Read the first string

mov eax, 3

mov ebx, 0

mov ecx, str1 ; Stores the string

mov edx, 4 ; Max bytes to read

int 0x80

; Read the second string

mov eax, 4

mov ebx, 1

mov ecx, prompt2

mov edx, prompt2_len

int 0x80

; Read the second string

mov eax, 3

mov ebx, 0

mov ecx, str2

mov edx, 4

int 0x80

; Initialize variables

xor ebx, ebx ; Bit difference count = 0

mov esi, str1 ; Address of str1

mov edi, str2 ; Address of str2

mov ecx, 3 ; Length of the strings

compare_loop:

mov al, [esi] ; Load byte from str1

```
xor al, [edi]    ; XOR with byte from str2
call count_bits ; Count set bits in AL
inc esi         ; Move to next byte in str1
inc edi         ; Move to next byte in str2
loop compare_loop ; Repeat for all bytes
```

```
; Convert bit count (EBX) to ASCII
add bl, '0'      ; Convert to ASCII ('0' + count)
mov [output], bl ; Store in output buffer
mov [output+1], byte 10 ; Add newline
```

```
; Print result
mov eax, 4
mov ebx, 1 ; stdout
mov ecx, output ; Buffer to print
mov edx, 2 ; Length
int 0x80
```

```
; Exit
mov eax, 1
xor ebx, ebx
int 0x80
```

```
; Count 1s in AL
count_bits:
    xor edx, edx ; Clear
bit_count_loop:
    test al, 1
    jz skip_inc
    inc edx      ; Increment count if bit is 1
```

skip_inc:

shr al, 1

jnz bit_count_loop ; Repeat if AL is not zero

add ebx, edx ; Add count to total

ret

```
[z151219@linux6 HamDisUpdated]$ a.out
Enter the first string (3 characters): foo
Enter the second string (3 characters): bar
8
[z151219@linux6 HamDisUpdated]$ a.out
Enter the first string (3 characters): max
Enter the second string (3 characters): mum
5
[z151219@linux6 HamDisUpdated]$
```